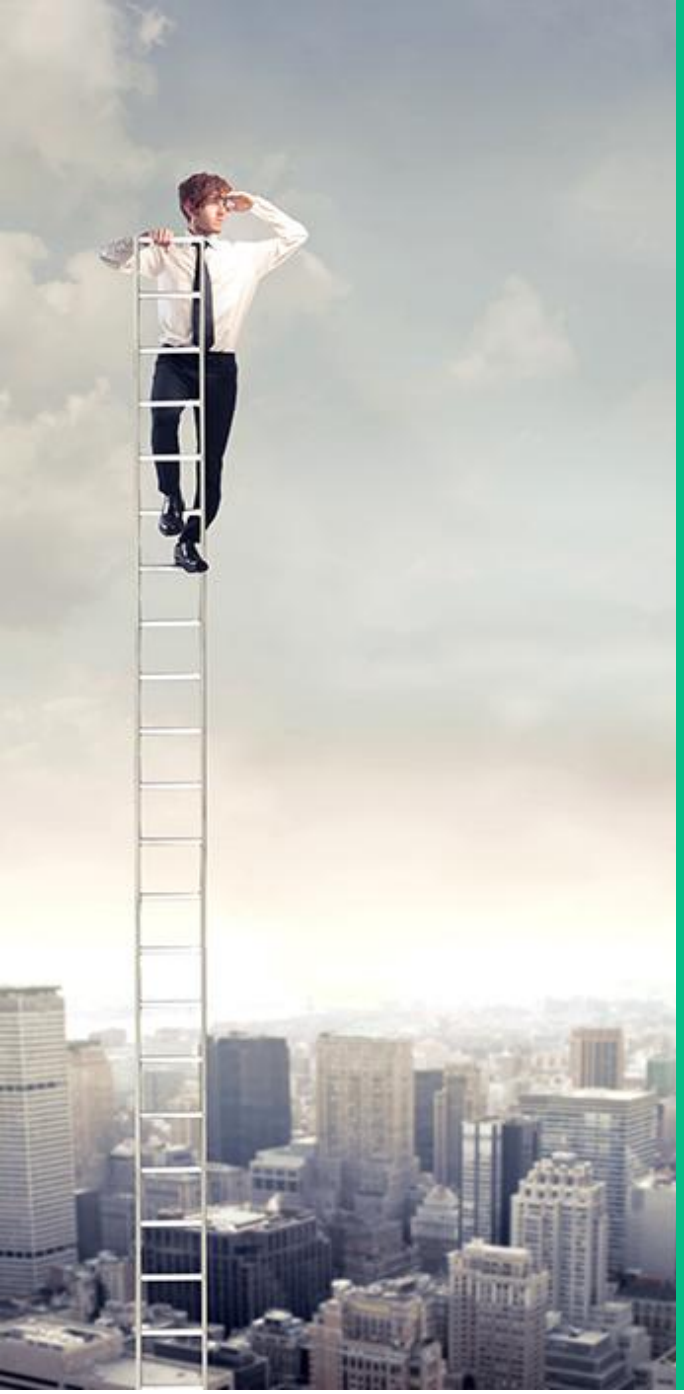


Natural Language Processing 03

16.10.2026

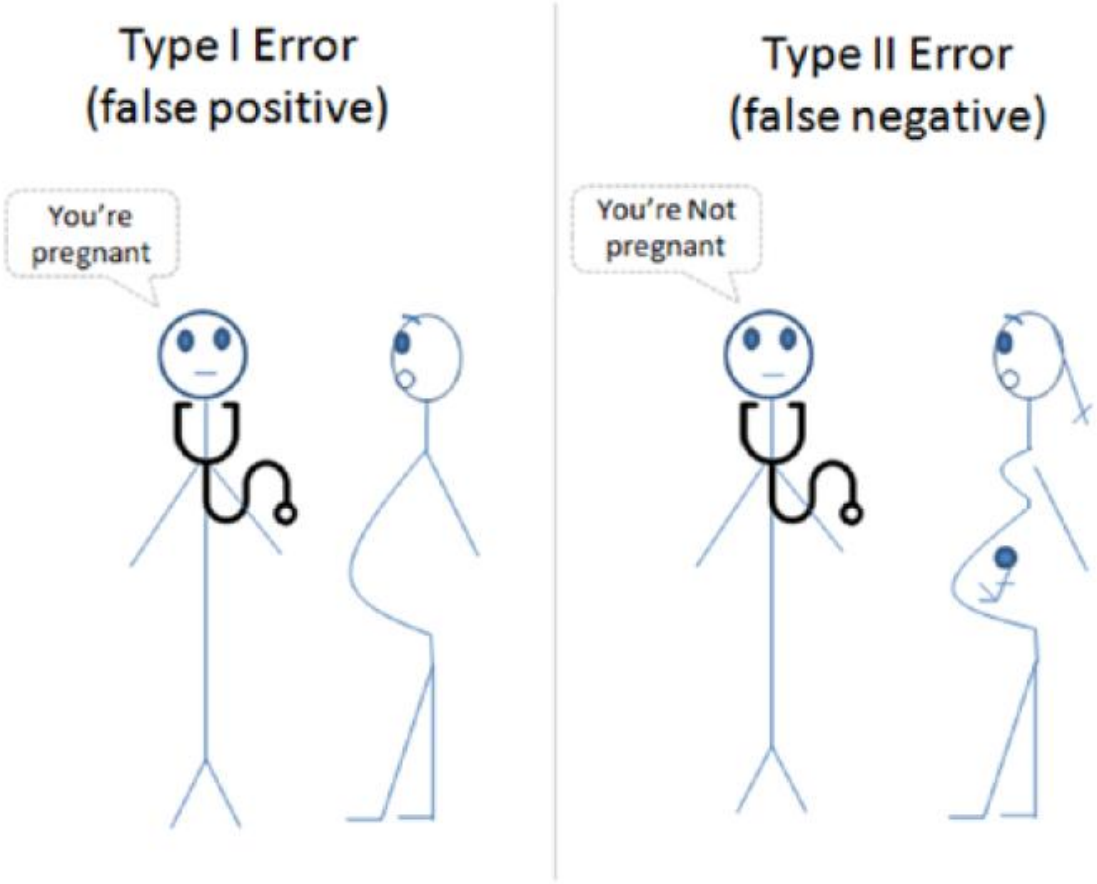
Czapiewski Paweł pczapiewski@pjwstk.edu.pl





Errors In ML

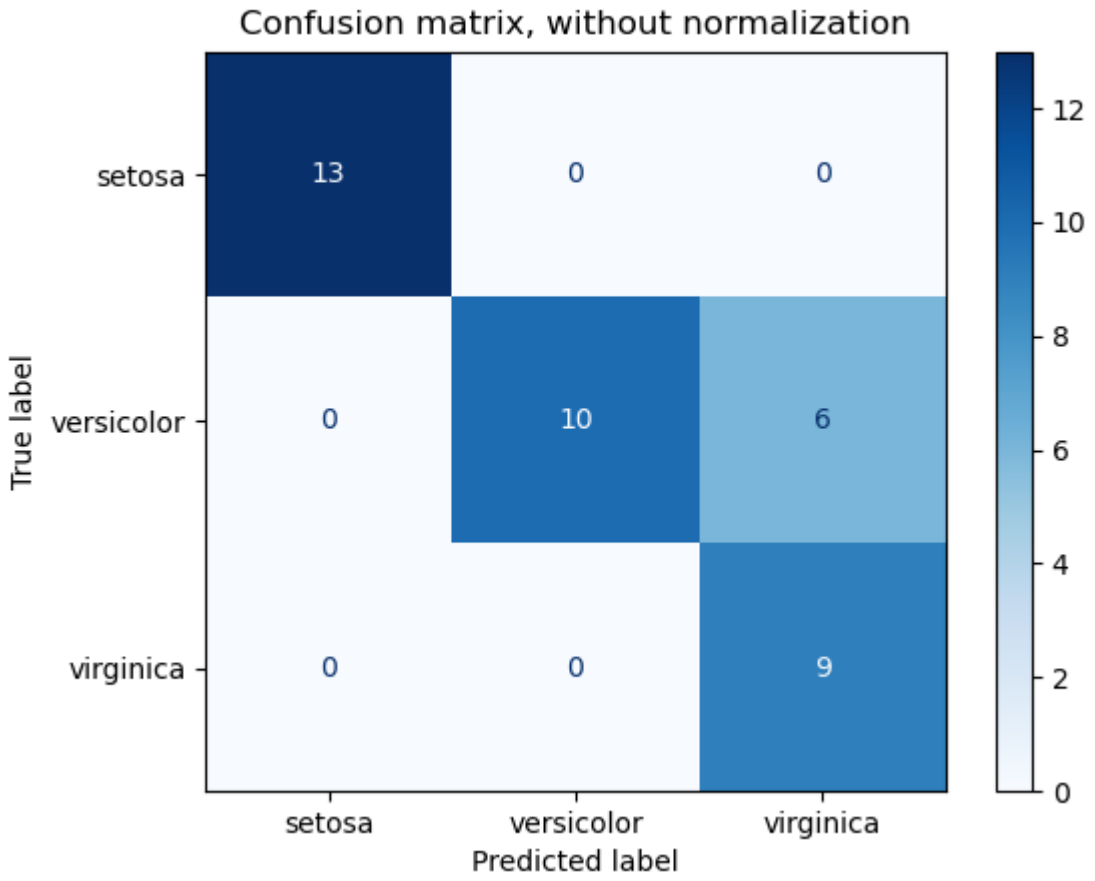
Errors in ML



Source: Mastering Machine Learning with Python in Six Steps

Confustion Matrix

```
                | Positive Prediction | Negative Prediction
Positive Class  | True Positive (TP)  | False Negative (FN)
Negative Class  | False Positive (FP) | True Negative (TN)
```



Source: https://scikit-learn.org/stable/auto_examples/model_selection/plot_confusion_matrix.html

Precision vs Recall

Precision
Precision is a metric that quantifies the number of correct positive predictions made.

Recall
Recall is a metric that quantifies the number of correct positive predictions made out of all positive predictions that could have been made.

Precision = TruePositives / (TruePositives + FalsePositives)

Recall = TruePositives / (TruePositives + FalseNegatives)

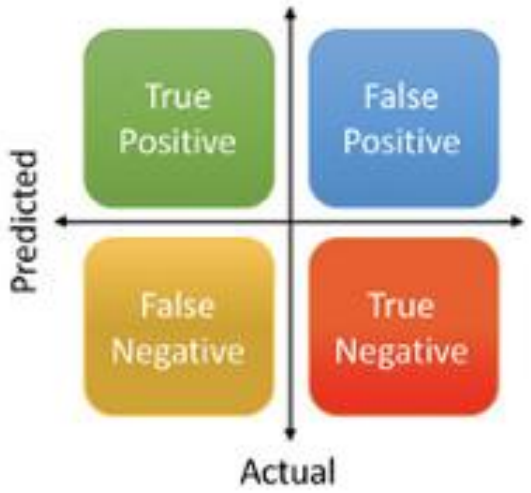
Source: https://machinelearningmastery.com/fbeta-measure-for-machine-learning/

Accuracy

$$\textbf{Precision} = \frac{\text{True Positive}}{\text{Actual Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive + False Positive}}$$

$$\textbf{Recall} = \frac{\text{True Positive}}{\text{Predicted Results}} \quad \text{or} \quad \frac{\text{True Positive}}{\text{True Positive + False Negative}}$$

$$\textbf{Accuracy} = \frac{\text{True Positive + True Negative}}{\text{Total}}$$



Source: https://medium.com/@shrutisaxena0617/precision-vs-recall-386cf9f89488

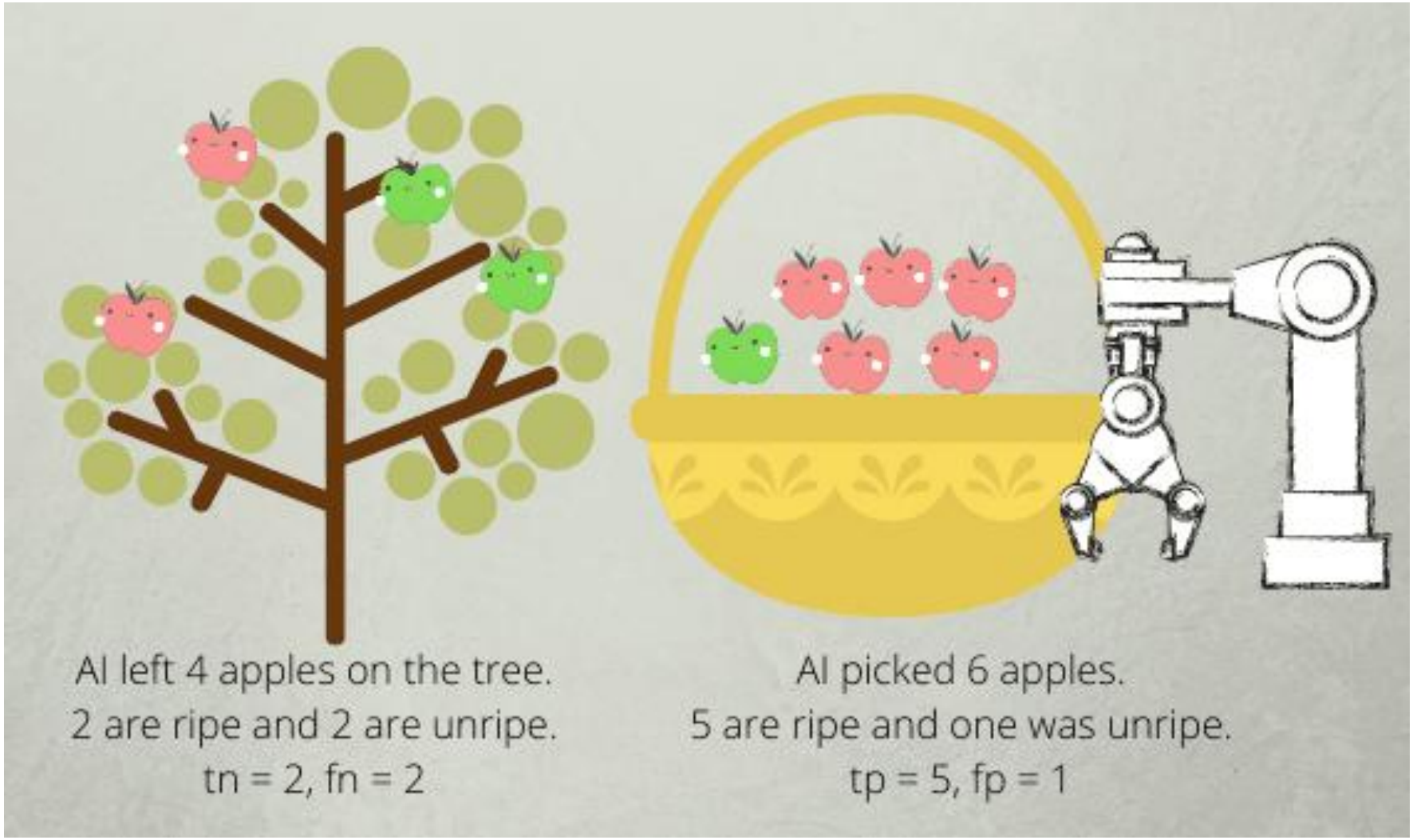
Generalized F_β-score Formula

$$F_\beta = (1 + \beta^2) \times \frac{\text{precision} \times \text{recall}}{(\beta^2 \times \text{precision}) + \text{recall}}$$
$$= \frac{(1 + \beta^2)\text{tp}}{(1 + \beta^2)\text{tp} + \beta^2\text{fn} + \text{fp})}$$

A factor indicating how much more important recall is than precision. For example, if we consider recall to be twice as important as precision, we can set *β* to 2. The standard F-score is equivalent to setting *β* to one.

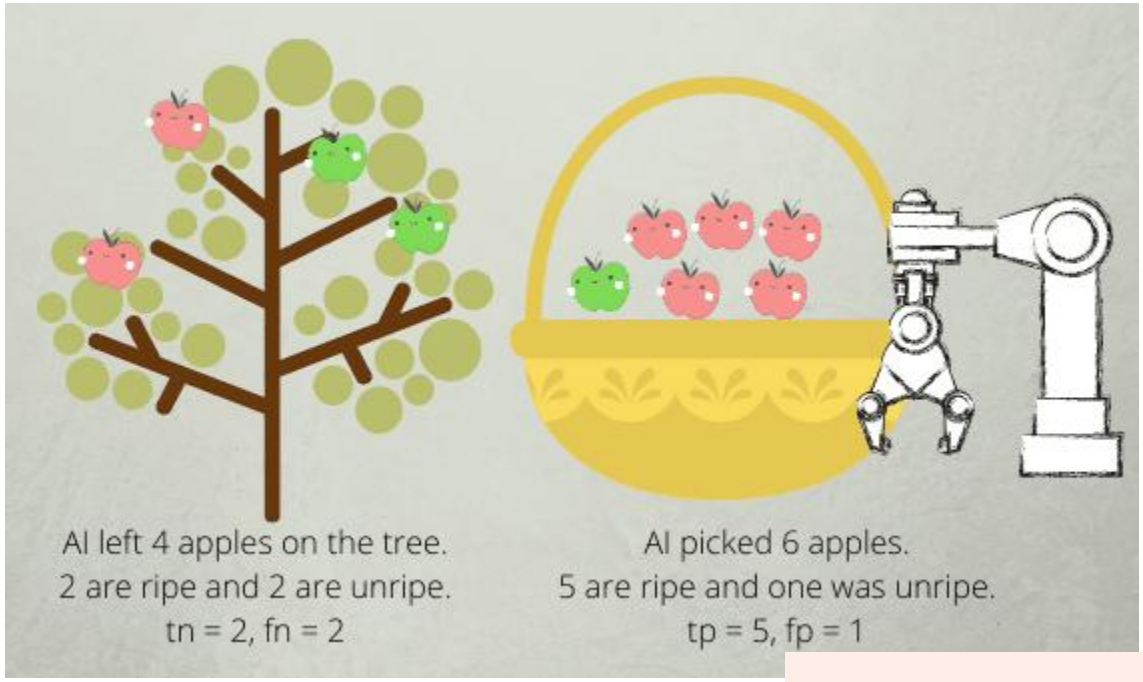
Source: https://deepai.org/machine-learning-glossary-and-terms/f-score

F-Score Calculation Example



Source: https://deepai.org/machine-learning-glossary-and-terms/f-score

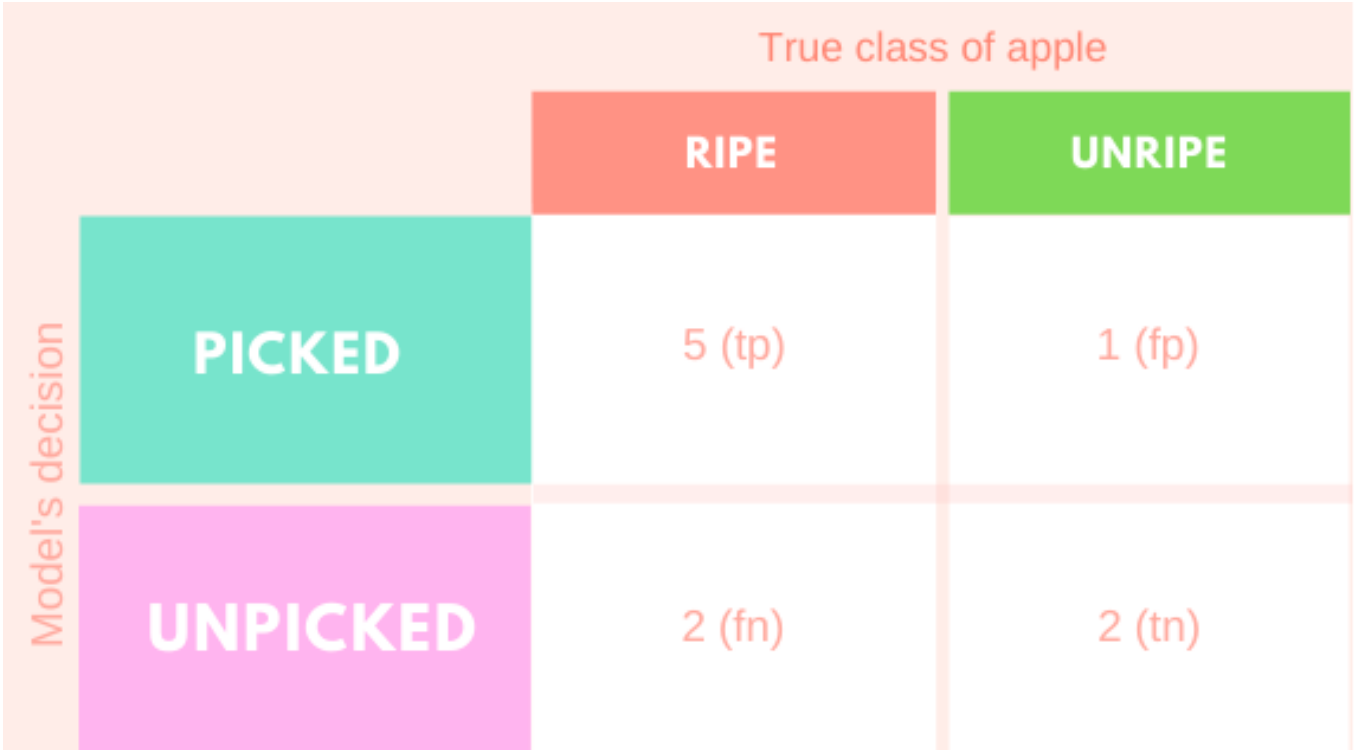
F-Score Calculation Example



Model's decision	True class of apple: RIPE	True class of apple: UNRIPE
PICKED	5 (tp)	1 (fp)
UNPICKED	2 (fn)	2 (tn)

Source: https://deepai.org/machine-learning-glossary-and-terms/f-score

F-Score Calculation Example

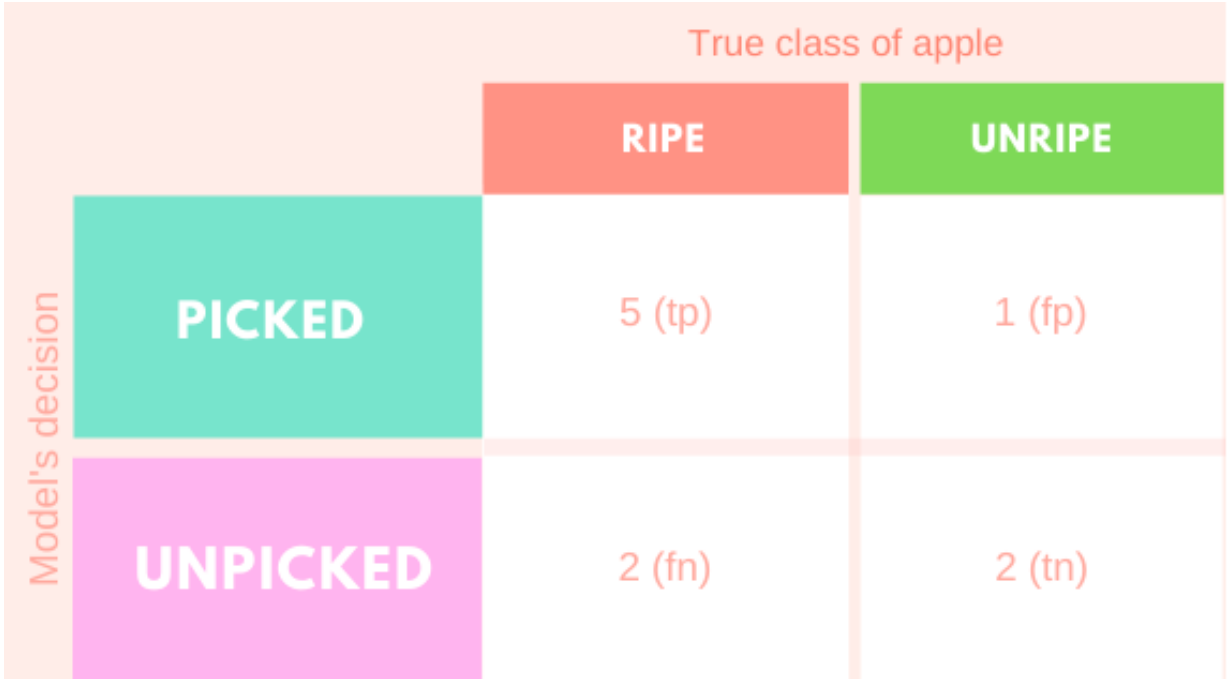


The model's precision is the number of ripe apples that were correctly picked, divided by all apples that the model picked.

$$\begin{aligned} \text{precision} &= \frac{\text{tp}}{\text{tp} + \text{fp}} \\ &= \frac{5}{5+1} \\ &= 0.83 \end{aligned}$$

Source: https://deepai.org/machine-learning-glossary-and-terms/f-score

F-Score Calculation Example

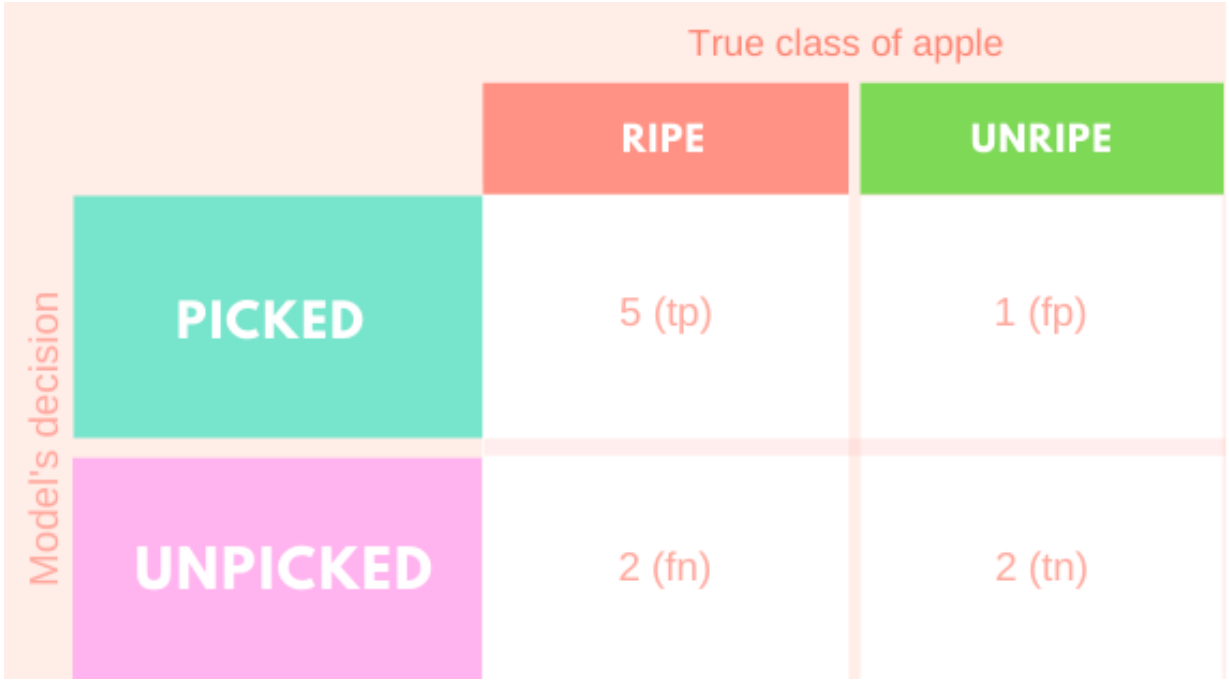


The recall is the number of ripe apples that were correctly picked, divided by the total number of ripe apples.

$$\begin{aligned} \text{recall} &= \frac{\text{tp}}{\text{tp} + \text{fn}} \\ &= \frac{5}{5 + 2} \\ &= 0.71 \end{aligned}$$

Source: https://deepai.org/machine-learning-glossary-and-terms/f-score

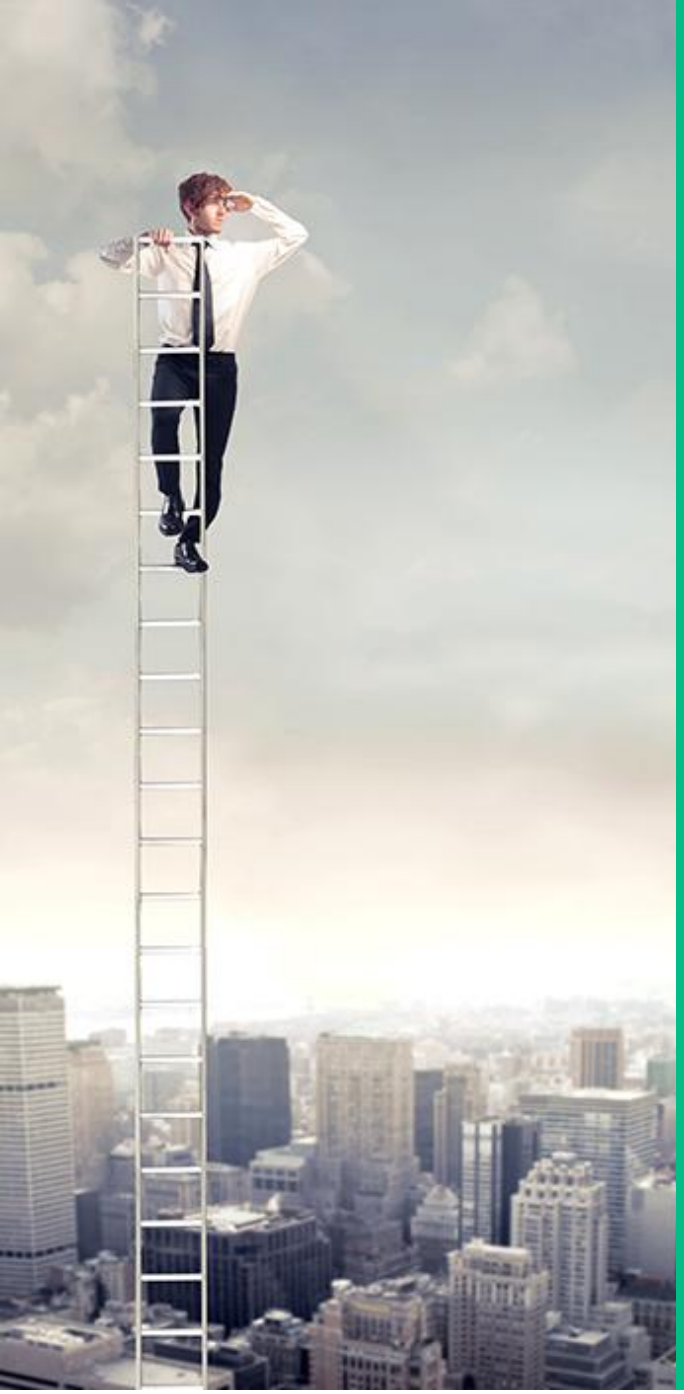
F-Score Calculation Example



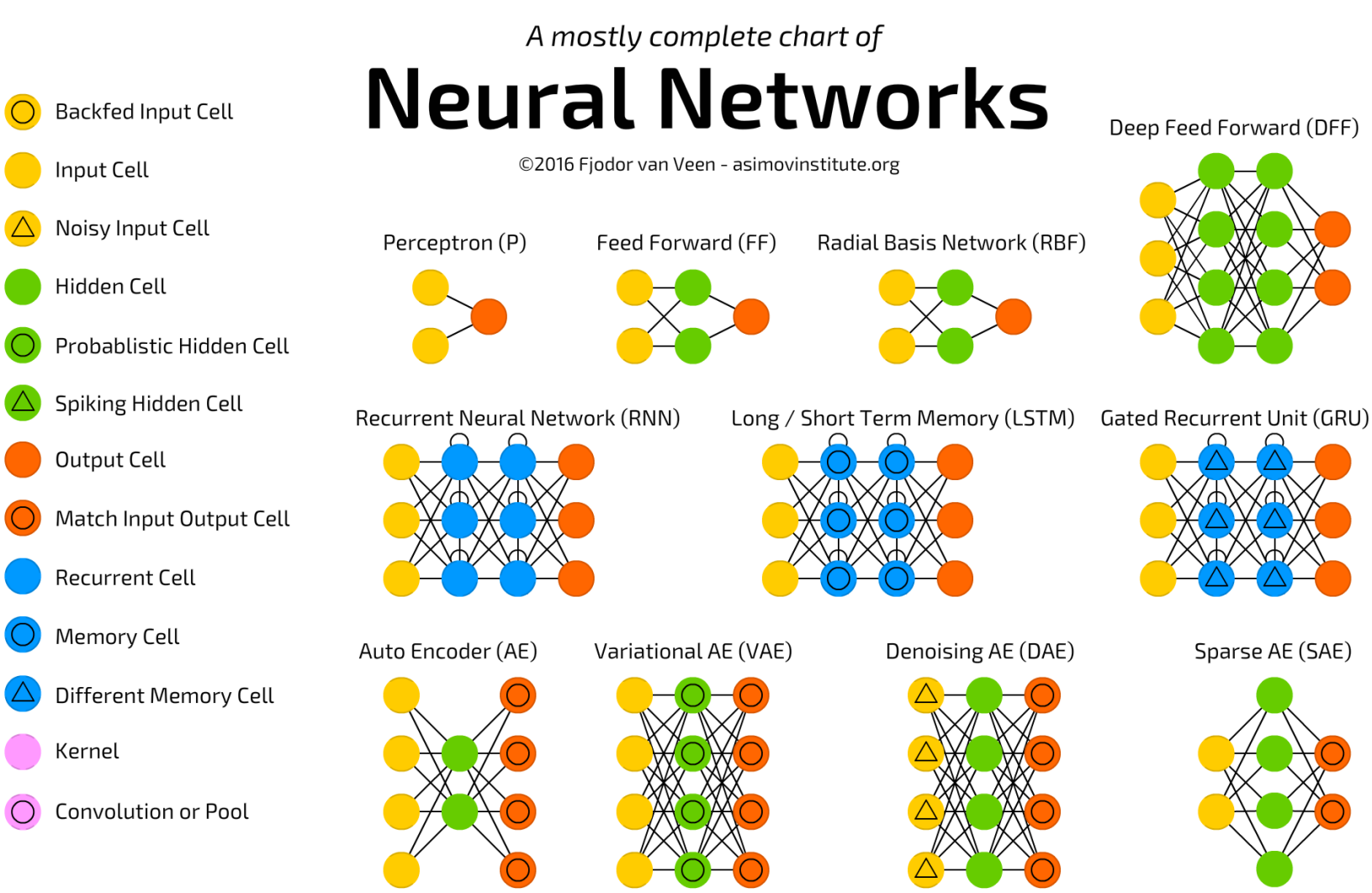
We can now calculate the F-score

$$
\begin{aligned}
F_1 &= 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \\
&= 2 \times \frac{0.83 \times 0.71}{0.83 + 0.71} \\
&= 0.77
\end{aligned}
$$

Source: https://deepai.org/machine-learning-glossary-and-terms/f-score

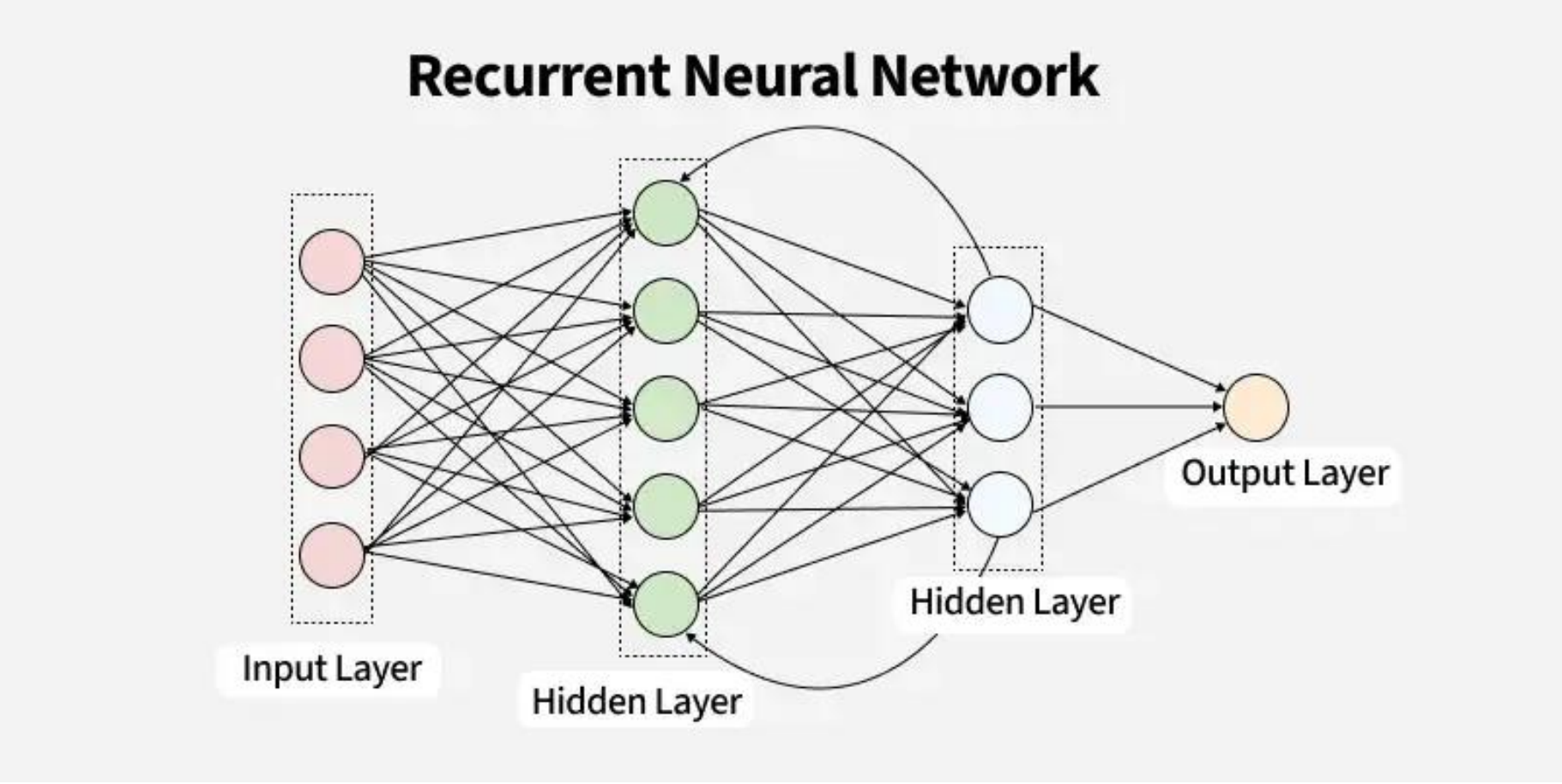


Recurrent Neural Network



Source: https://towardsdatascience.com/the-mostly-complete-chart-of-neural-networks-explained-3fb6f2367464

Introduction to Recurrent Neural Networks



Source: https://www.geeksforgeeks.org/machine-learning/introduction-to-recurrent-neural-network/

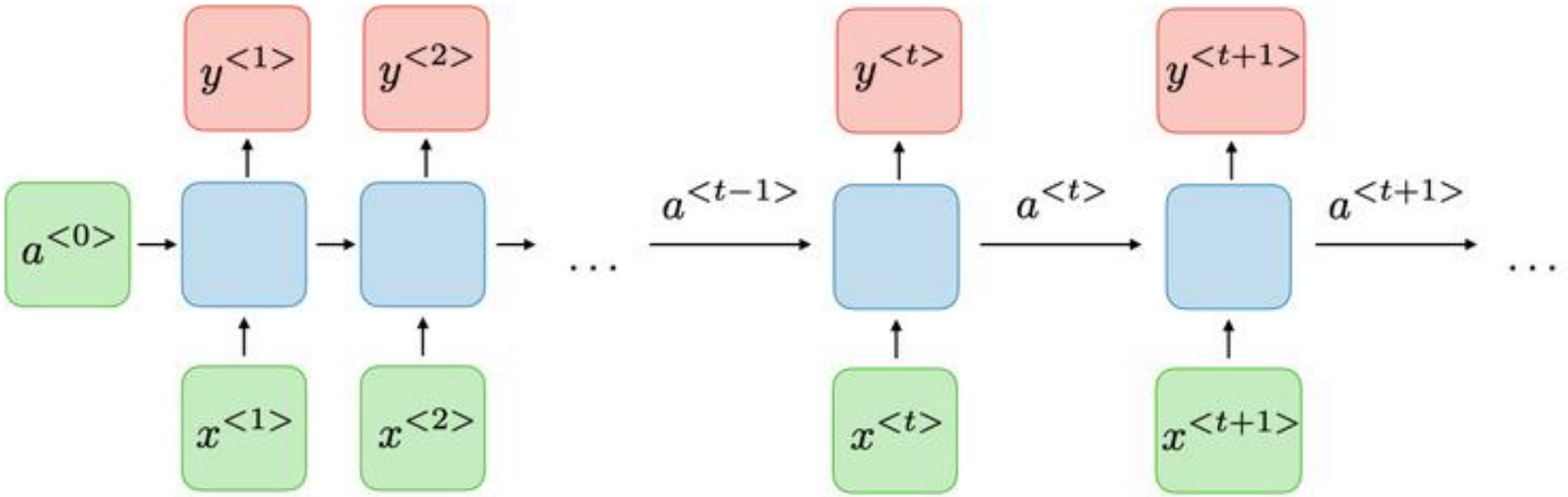
Introduction to Recurrent Neural Networks

ASPECT	CNN (CONVOLUTIONAL NEURAL NETWORK)	RNN (RECURRENT NEURAL NETWORK)
Data Type	Grid-like data (e.g., images, videos).	Sequential data (e.g., time series, speech, text).
Architecture	Layers of convolution filters that process data spatially.	Recurrence structure that processes data temporally, step-by-step.
Focus	Extracts spatial features from data.	Extracts temporal or sequential dependencies.
Memory	No explicit memory; processes entire input at once.	Retains a hidden state to "remember" past inputs.
Applications	Image classification, object detection, facial recognition.	Natural language processing, time-series analysis, speech tasks.
Training	Easier to parallelize due to independent operations.	Harder to parallelize, as sequential dependencies require each step to be computed before the next.

Source: https://medium.com/@hassaanidrees7/cnn-vs-rnn-understanding-their-roles-in-image-and-sequential-data-processing-65684cc05902

Introduction to Recurrent Neural Networks

Architecture of a traditional RNN Recurrent neural networks, also known as RNNs, are a class of neural networks that allow previous outputs to be used as inputs while having hidden states. They are typically as follows:



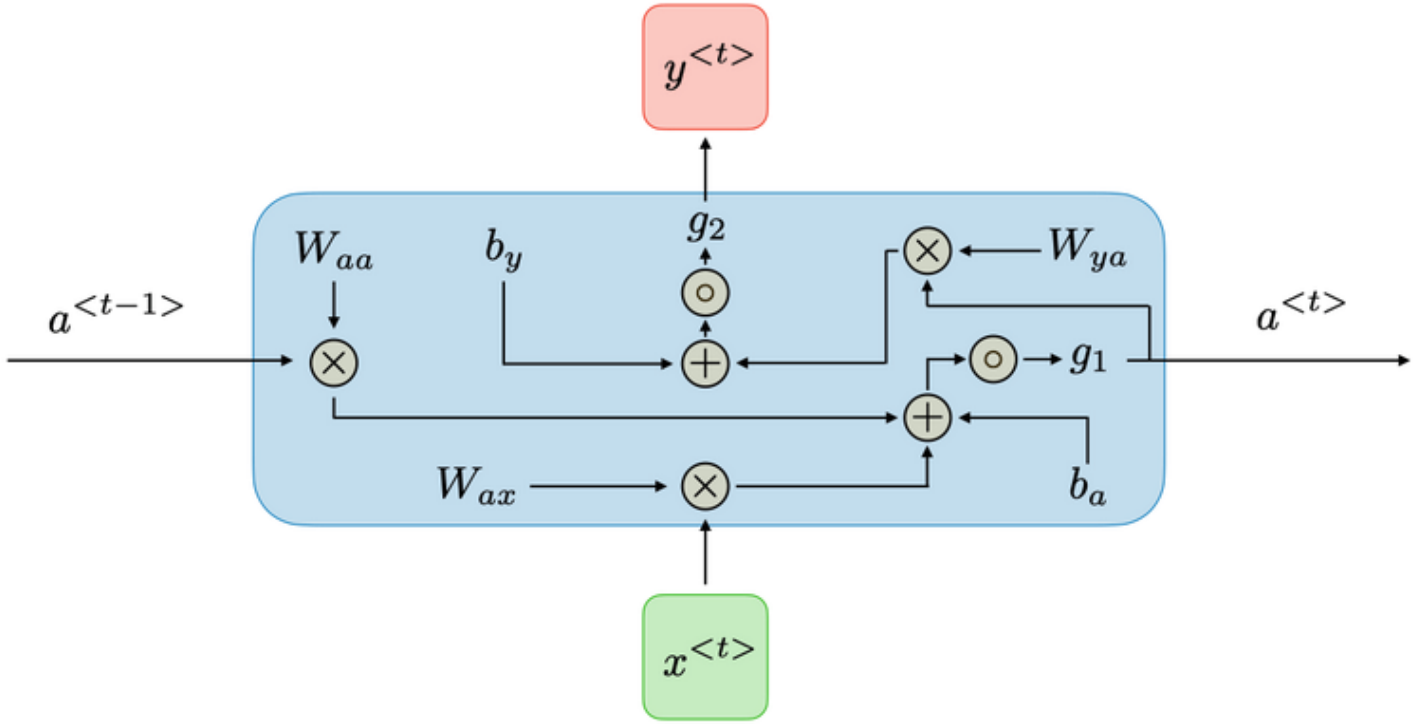
Source: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

Introduction to Recurrent Neural Networks

For each timestep t, the activation $a^{<t>}$ and the output $y^{<t>}$ are expressed as follows:

$$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a) \quad \text{and} \quad y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$$

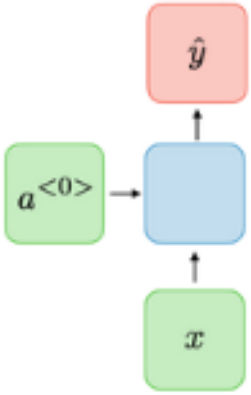
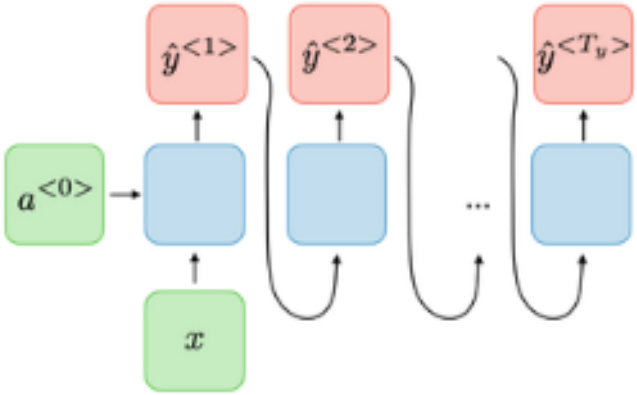
where $W_{ax}, W_{aa}, W_{ya}, b_a, b_y$ are coefficients that are shared temporally and g_1, g_2 activation functions.



Source: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

Introduction to Recurrent Neural Networks

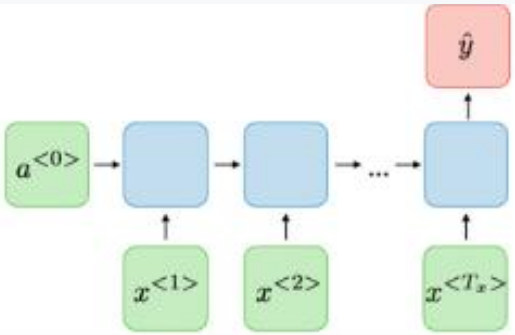
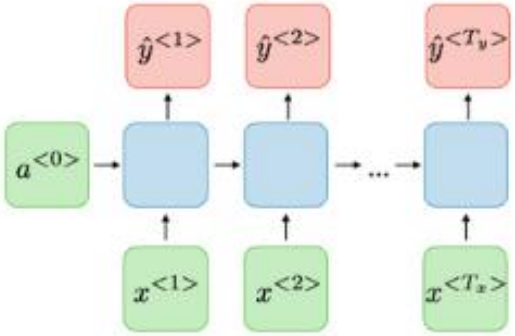
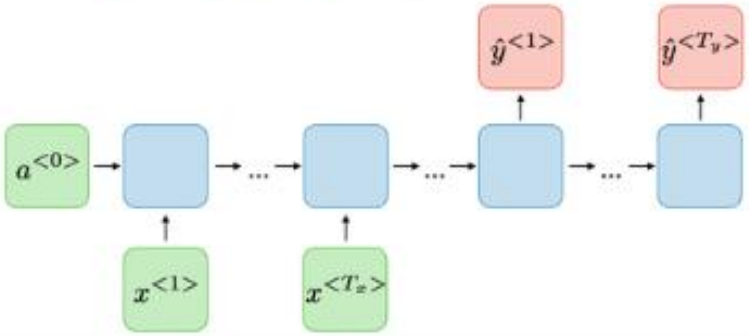
Applications of RNNs

Type of RNN	Illustration	Example
One-to-one $T_x = T_y = 1$		Traditional neural network
One-to-many $T_x = 1, T_y > 1$		Music generation

Source: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

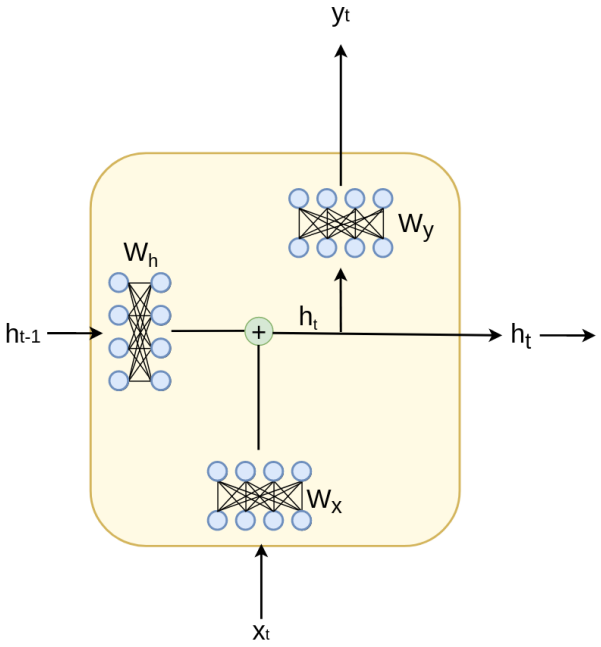
Introduction to Recurrent Neural Networks

Applications of RNNs

Type of RNN	Illustration	Example
Many-to-one $T_x > 1, T_y = 1$		Sentiment classification
Many-to-many $T_x = T_y$		Name entity recognition
Many-to-many $T_x \neq T_y$		Machine translation

Source: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks

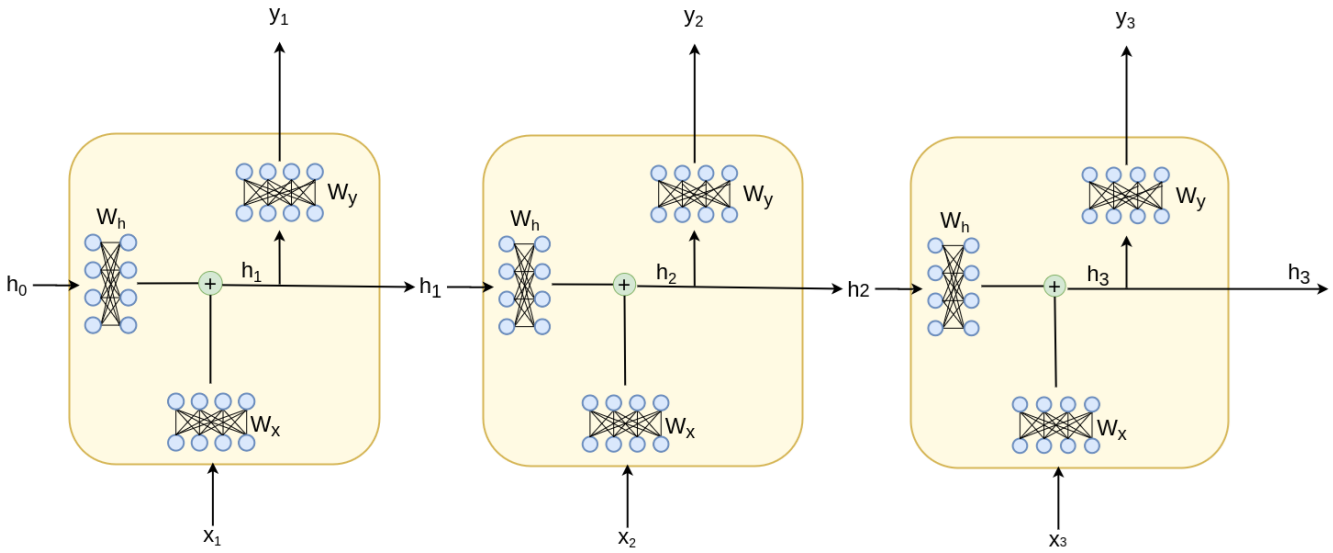
RNN Cell



$$h_t = \sigma_h(W_x x_t + W_h h_{t-1})$$

$$y = \sigma_y(W_y h_t)$$

Propagacja błędu dla W_y



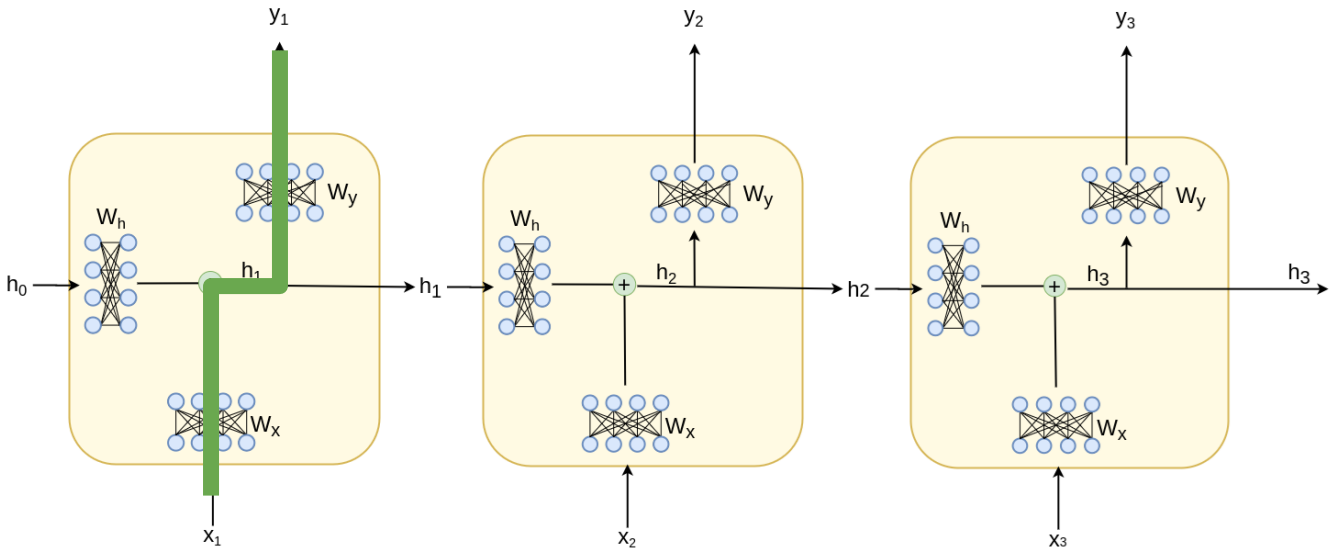
$$\frac{\delta L_1}{\delta W_y} = \frac{\delta L_1}{\delta \hat{y_1}} \frac{\delta y_1}{\delta W_y}$$

$$\frac{\delta L_2}{\delta W_y} = \frac{\delta L_2}{\delta \hat{y_2}} \frac{\delta y_2}{\delta W_y}$$

$$\frac{\delta L_3}{\delta W_y} = \frac{\delta L_3}{\delta \hat{y_3}} \frac{\delta y_3}{\delta W_y}$$

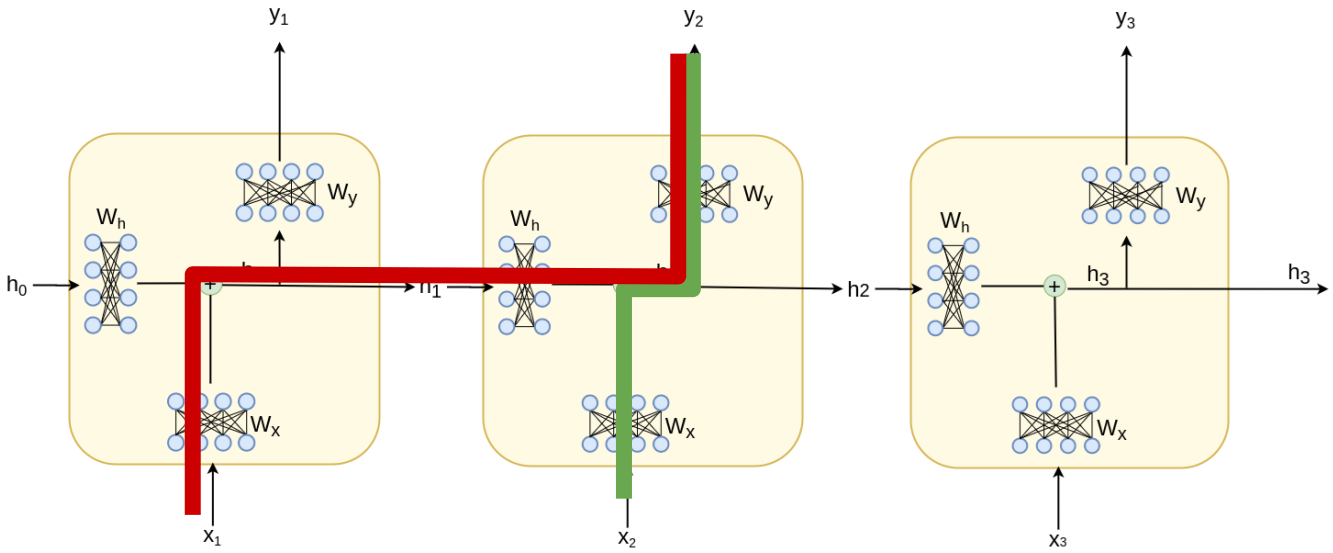
$$\frac{\delta L}{\delta W_y} = \frac{\delta L_1}{\delta \hat{y_1}} \frac{\delta y_1}{\delta W_y} + \frac{\delta L_2}{\delta \hat{y_2}} \frac{\delta y_2}{\delta W_y} + \frac{\delta L_3}{\delta \hat{y_3}} \frac{\delta y_3}{\delta W_y}$$

Propagacja błędu dla W_y



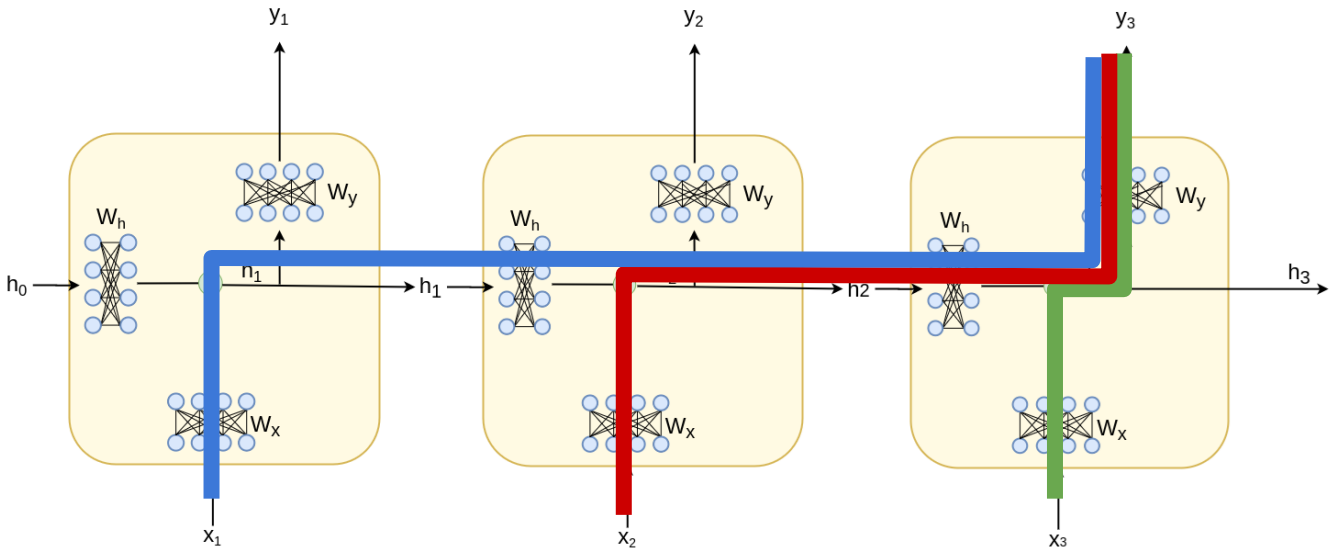
$$\frac{\delta L_1}{\delta W_x} = \frac{\delta L_1}{\delta \hat{y}_1} \frac{\delta \hat{y}_1}{\delta h_1} \frac{\delta h_1}{W_x}$$

Propagacja błędu dla W_y



$$\frac{\delta L_2}{\delta W_x} = \frac{\delta L_2}{\delta \hat{y}_2}\frac{\delta \hat{y}_2}{\delta h_2}\frac{\delta h_2}{W_x} + \frac{\delta L_2}{\delta \hat{y}_2}\frac{\delta \hat{y}_2}{\delta h_2}\frac{\delta h_2}{\delta h_1}\frac{\delta h_1}{\delta W_x}$$

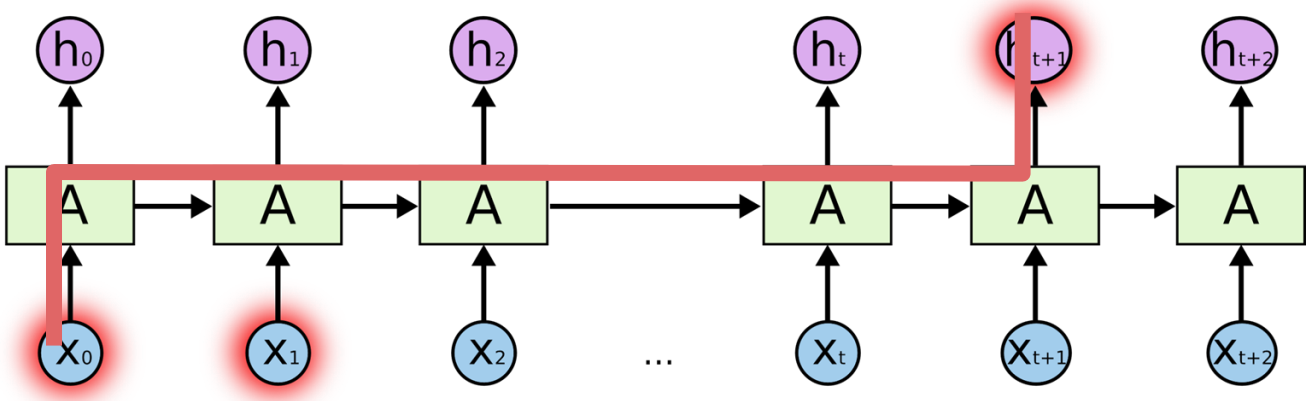
Propagacja błędu dla W_y



$$\frac{\delta L_3}{\delta W_x} = \frac{\delta L_3}{\delta \hat{y}_3}\frac{\delta \hat{y}_3}{\delta h_3}\frac{\delta h_3}{W_x} + \frac{\delta L_3}{\delta \hat{y}_3}\frac{\delta \hat{y}_3}{\delta h_3}\frac{\delta h_3}{\delta h_2}\frac{\delta h_2}{\delta W_x} + \frac{\delta L_3}{\delta \hat{y}_3}\frac{\delta \hat{y}_3}{\delta h_3}\frac{\delta h_3}{\delta h_2}\frac{\delta h_2}{\delta h_1}\frac{\delta h_1}{\delta W_x}$$

Vanishing/exploding gradient

The **vanishing** and **exploding** gradient phenomena are often encountered in the context of RNNs.



The reason why they happen is that it is difficult to capture long term dependencies because of multiplicative gradient that can be exponentially decreasing/increasing with respect to the number of layers.

Introduction to Recurrent Neural Networks

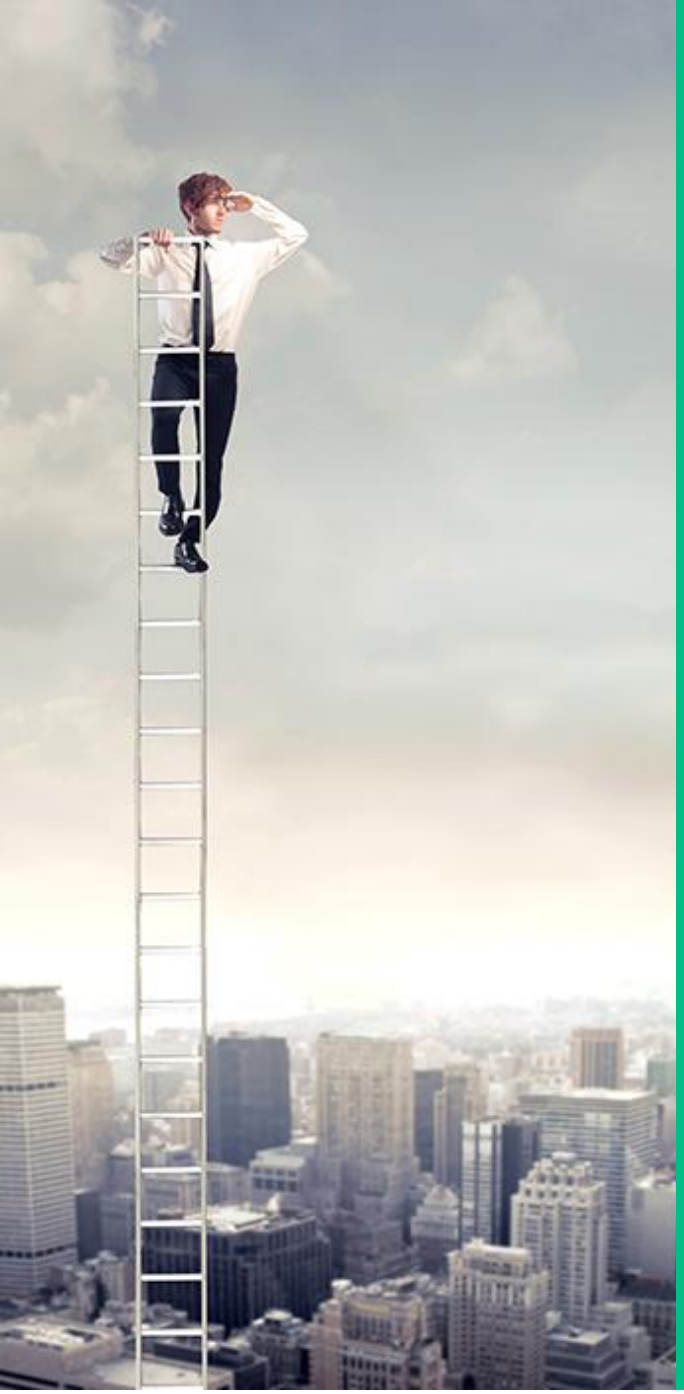
❒ **Loss function** — In the case of a recurrent neural network, the loss function $\mathcal{L}$ of all time steps is defined based on the loss at every time step as follows:

$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$$

❒ **Backpropagation through time** — Backpropagation is done at each point in time. At timestep T, the derivative of the loss $\mathcal{L}$ with respect to weight matrix W is expressed as follows:

$$\frac{\partial \mathcal{L}^{(T)}}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}^{(T)}}{\partial W}\Big|_{(t)}$$

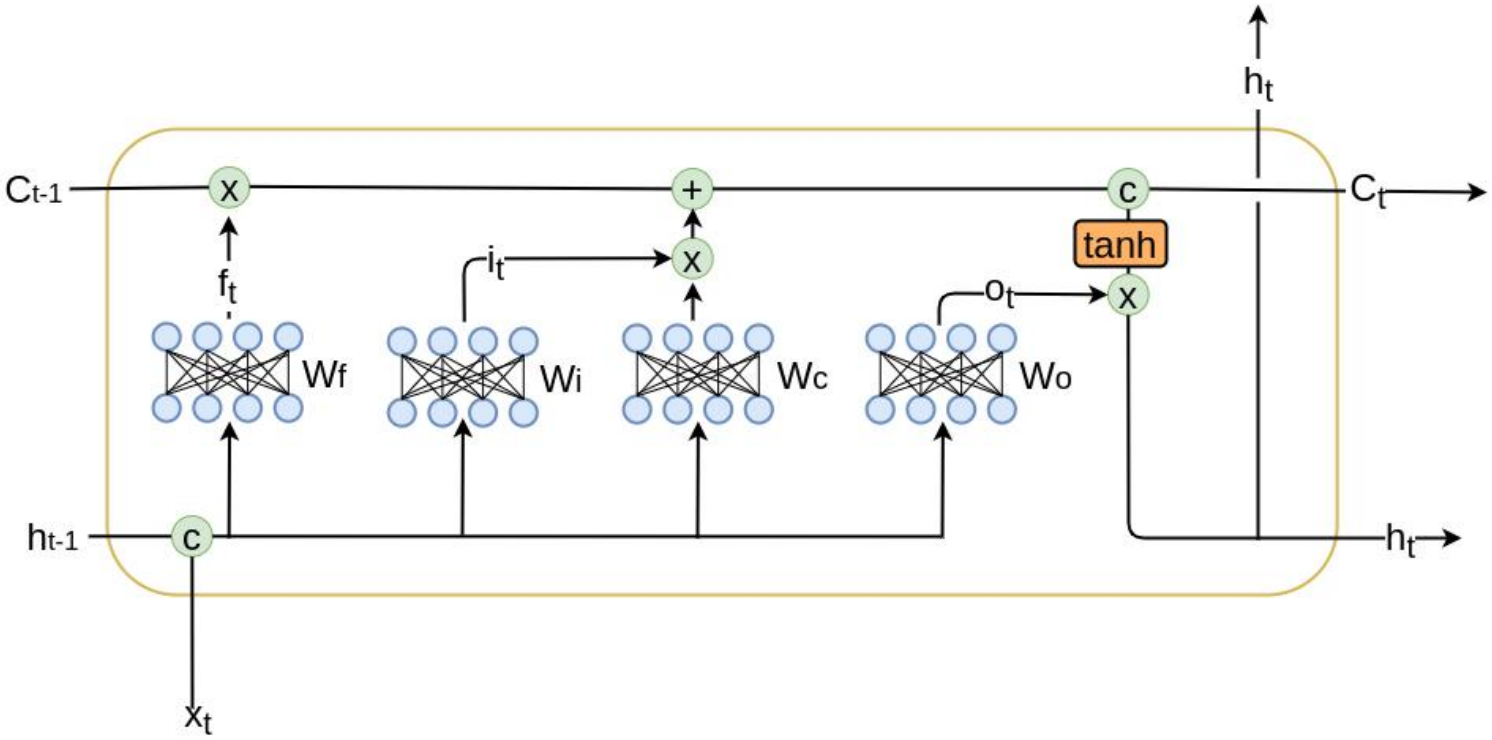
Source: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks





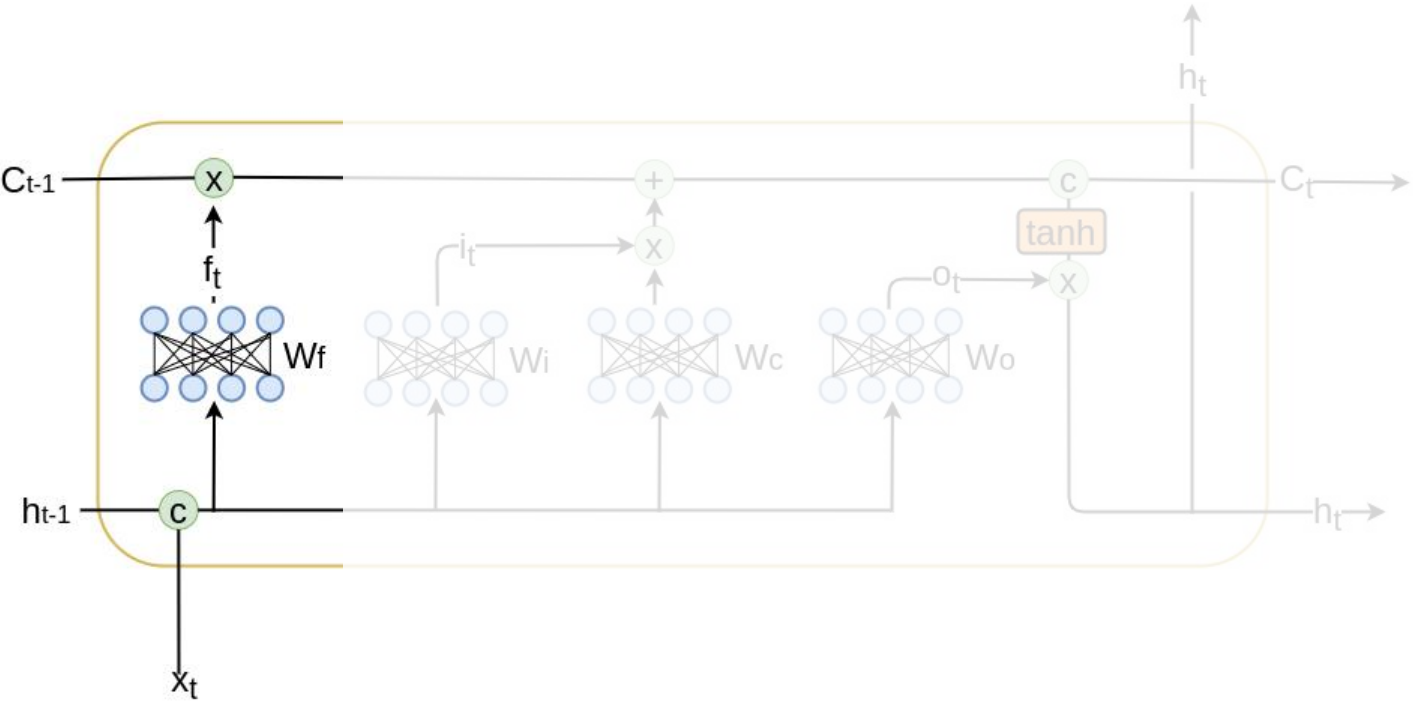
LSTM Networks

LSTM - Cell



LSTM - Cell

Bramka zapominająca

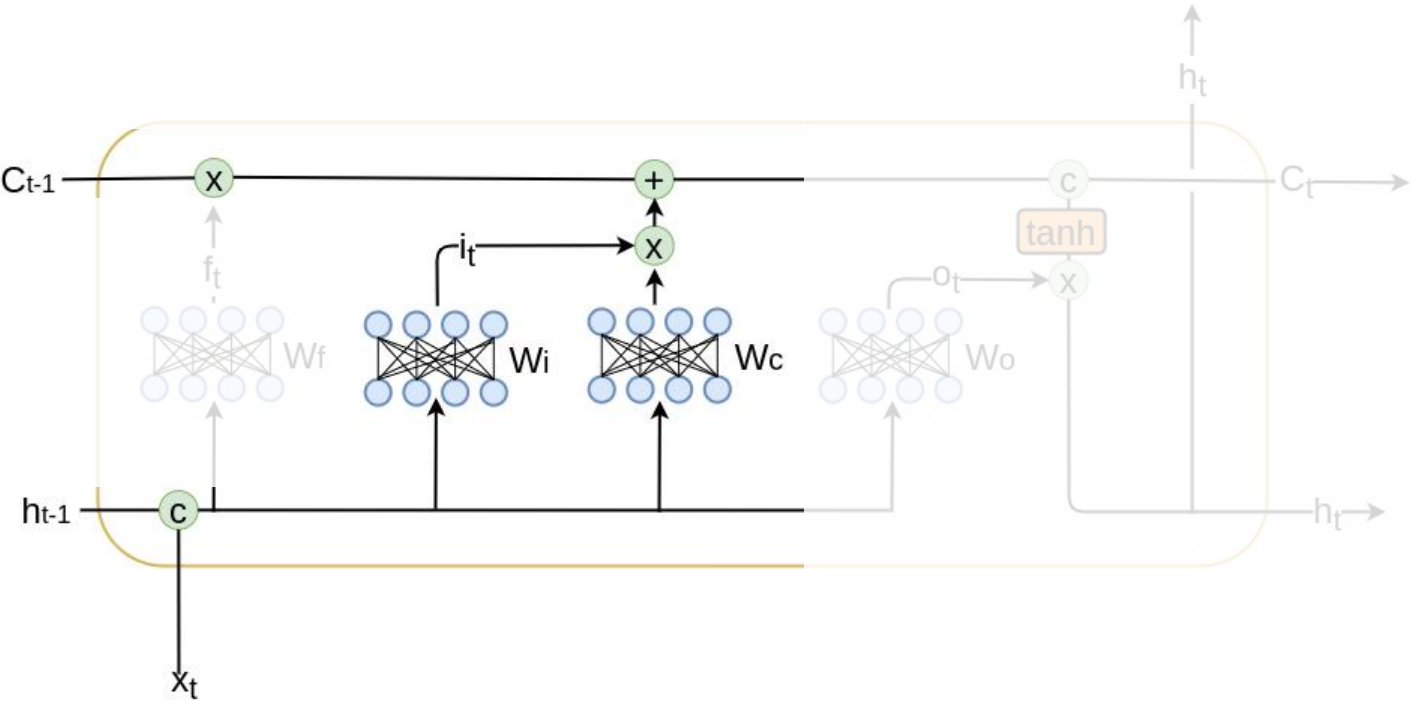


- Decyduje na podstawie pamięci krótkoterminowej (h) i wejścia (x) co zapomnieć z pamięci długoterminowej (c)
- "Zapominanie" odbywa się poprzez mnożenie wektorów

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

LSTM - Cell

Bramka wejściowa



Decyduje co z wejścia i pamięci krótkotrwałej dodać do pamięci długotrwałej

W_c decyduje "co dodać"
W_i decyduje "jak bardzo dodać"

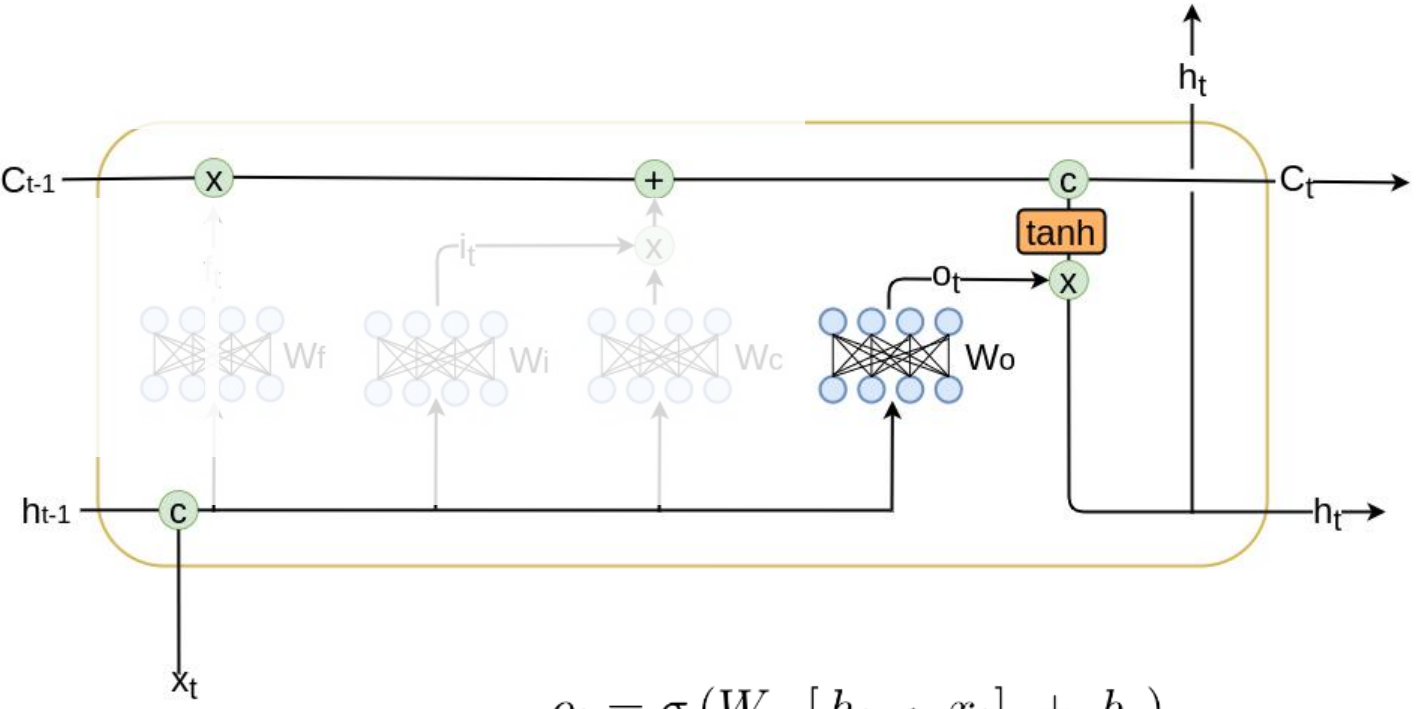
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM - Cell

Bramka wyjściowa



Oblicza wyjście i pamięć krótkotrwałą (h) na podstawie pamięci długotrwałej ©

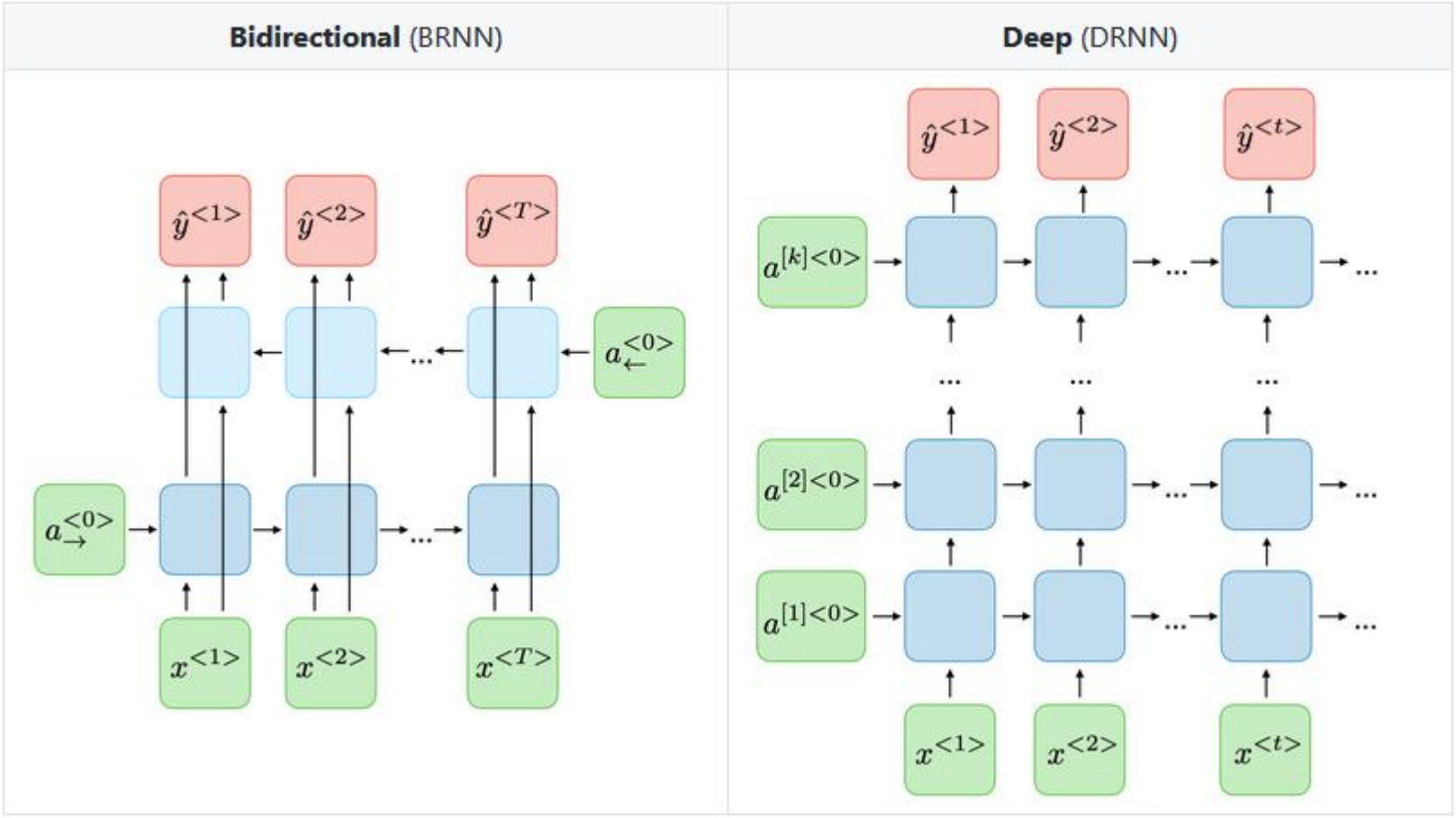
Decyduje na podstawie wejścia i pamięci krótkotrwałej co z przekazać do wyjścia

$$o_t = \sigma\left(W_o\ [h_{t-1}, x_t]\ +\ b_o\right)$$

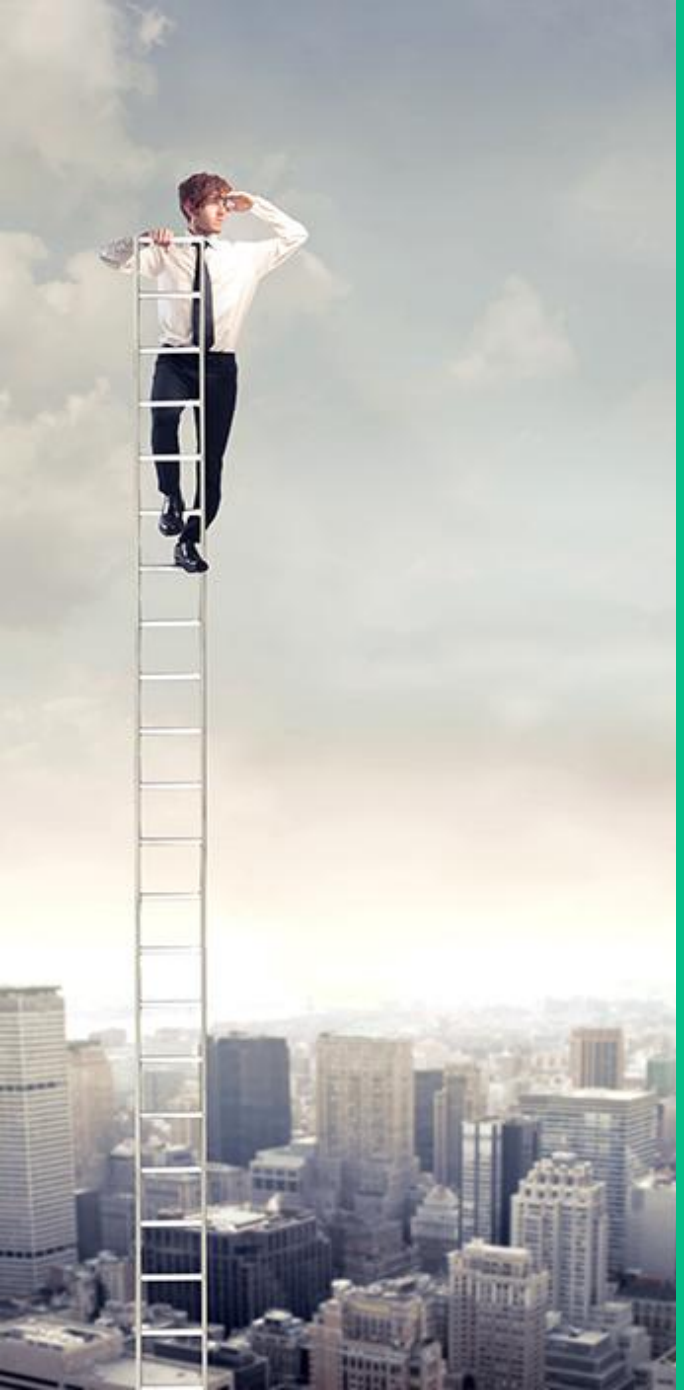
$$h_t = o_t * \tanh\left(C_t\right)$$

Variants of RNNs

The table below sums up the other commonly used RNN architectures:



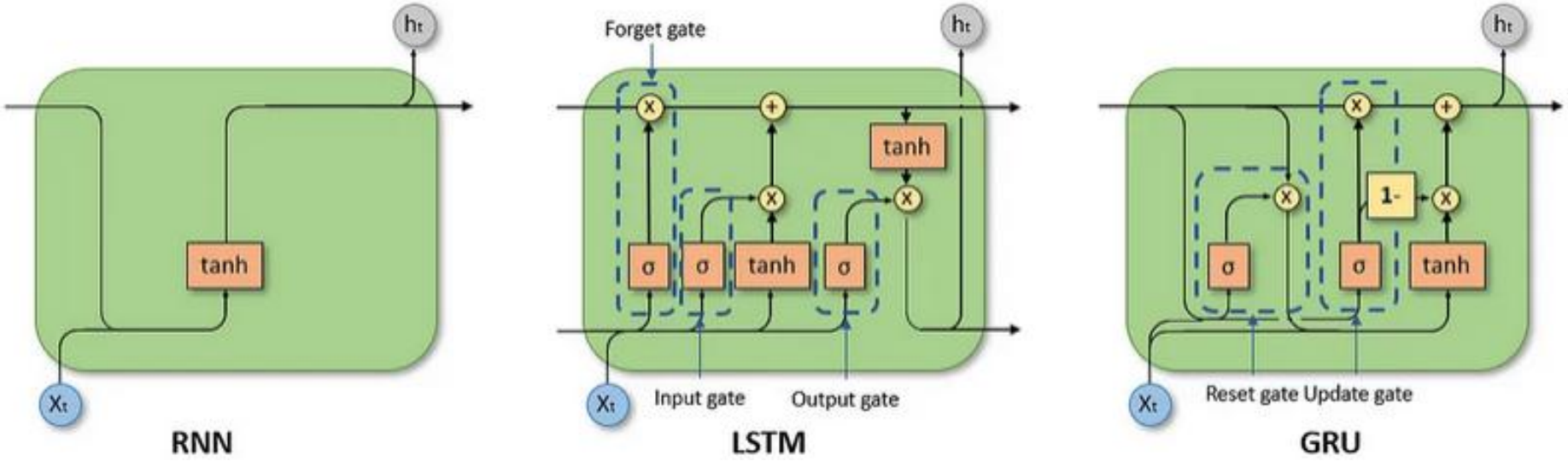
Source: https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-recurrent-neural-networks



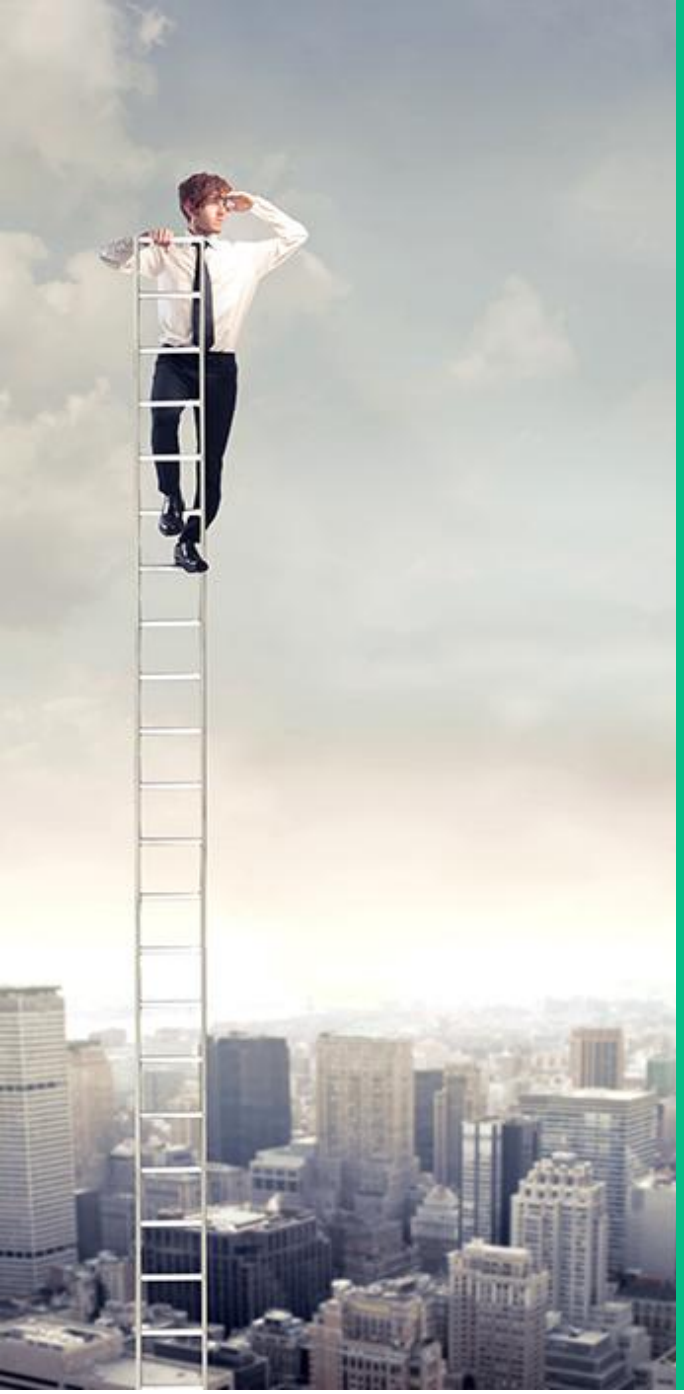


GRU Networks

Gated Recurrent Unit



Source: https://towardsdatascience.com/a-brief-introduction-to-recurrent-neural-networks-638f64a61ff4





Thank you for attention!